

ECE 0322.1

MICROPROCESSOR AND MICROCONTROLLER SYSTEM AND DESIGN

2nd Semester, S.Y. 2025–2026

**Design and Implementation of a
Scalable IoT-Enabled
Smart Parking Management System**

Final Technical Report

Submitted by:

GROUP 6

Calvan | Dela Cruz | Loyola

Omandac | Salde | Solinap

Submitted to:

Engr. Fernando Victor V. De Vera

Course Instructor

May 2026

Abstract

This paper presents the design, implementation, and evaluation of a Scalable IoT-Enabled Smart Parking Management System developed as the final project for ECE 0322.1. The system addresses the manual, error-prone process of locating available parking slots in commercial and residential parking facilities by providing real-time occupancy monitoring, remote slot reservation, and automated entry/exit gate control. The prototype integrates an ESP32 WROOM-32 microcontroller, two HC-SR04 ultrasonic sensors, an SG90 servo motor, an I²C 16×2 LCD, and per-slot indicator LEDs and limit switches. A pair of 74HC595 output shift registers and a pair of 74HC165 input shift registers, both daisy-chained, provide expandable input/output capacity to accommodate up to eight parking slots using only six microcontroller GPIO pins. Wireless connectivity is established through the ESP32's built-in 2.4 GHz Wi-Fi module, and bidirectional cloud synchronization is achieved through Firebase Realtime Database. A companion web application, hosted on Firebase Hosting, enables remote monitoring, reservation, and alert delivery. Functional testing across six representative scenarios confirmed correct operation of all primary subsystems with sub-second response latency. The discussion concludes with a deployment roadmap demonstrating how the prototype's modular architecture supports transition to a full-scale parking facility through component substitution (industrial-grade sensors, PoE-powered controllers, multi-node mesh networking), with projected scaling to several hundred slots per controller node.

Keywords: *Internet of Things (IoT), embedded systems, ESP32, shift register, Firebase, scalability, smart parking, real-time monitoring.*

1. Introduction

1.1 Background

The rapid growth of urban populations in the Philippines and across Southeast Asia has placed significant pressure on parking infrastructure in commercial, residential, and institutional settings. Drivers searching for available parking spaces contribute disproportionately to local traffic congestion, fuel consumption, and emissions. Conventional parking facilities provide little or no real-time information regarding slot availability, requiring drivers to navigate the facility manually before discovering whether space is available. Furthermore, the absence of remote reservation mechanisms means that arriving drivers have no guarantee that an advertised vacancy will still be available upon arrival.

The Internet of Things (IoT) paradigm offers a compelling solution to this class of problem. By instrumenting individual parking slots with low-cost occupancy sensors and connecting the resulting data stream to a cloud platform, real-time information can be made available to drivers, facility operators, and automated systems. The same network can also support remote reservation, alert notification, and historical analytics.

1.2 Problem Statement

Traditional parking facilities lack three capabilities that are central to modern smart-city infrastructure:

1. **Real-time occupancy visibility** — drivers and operators cannot determine current slot status without physically inspecting each space.
2. **Remote reservation** — there is no mechanism by which a driver can secure a slot prior to arrival.
3. **Automated access control with state awareness** — gate operation is typically untied to the actual occupancy state of the facility, leading to redundant manual checks.

This project addresses these gaps through the design of a microcontroller-based IoT system that integrates sensing, cloud synchronization, remote user interfaces, and automated actuation into a single coherent prototype.

1.3 Objectives

The general objective of this project is to design, construct, and evaluate a functional IoT-enabled smart parking management system that demonstrates real-time monitoring, remote control, and scalability.

The specific objectives are as follows:

4. To design a hardware architecture using an ESP32 microcontroller and daisy-chained shift registers capable of supporting at least eight parking slots through six GPIO pins;
5. To develop embedded firmware that continuously monitors slot occupancy, synchronizes state with a cloud database, and responds to remote reservation commands;
6. To implement a web-based dashboard for remote monitoring, slot reservation, and event alerts;
7. To validate functional correctness through systematic test scenarios; and

8. To formulate a deployment roadmap describing the transition from laboratory prototype to full-scale commercial installation.

1.4 Significance of the Study

The output of this study contributes a reproducible, low-cost reference design for IoT-enabled parking management. The modular hardware architecture, in which slot count is decoupled from microcontroller pin count, directly addresses the scalability requirement that distinguishes IoT-class systems from conventional embedded controllers. By documenting both the prototype implementation and the migration path to industrial deployment, the project also serves as an academic vehicle for demonstrating the end-to-end design lifecycle expected in modern embedded systems engineering.

1.5 Scope and Limitations

The prototype constructed for this project is a tabletop miniature representing a four-slot parking facility (with provision for up to eight slots in firmware). The scope and limitations are as follows:

- **Included:** Real-time occupancy detection through mechanical limit switches; automated gate control through ultrasonic sensors and a servo motor; cloud synchronization through Firebase Realtime Database; a responsive web dashboard with authentication; per-slot bicolor LED indicators; and a 16×2 LCD summary display.
- **Not included:** Outdoor weatherproofing; vehicle license-plate recognition; payment processing; integration with municipal traffic systems; battery backup; and lightning protection. These are addressed analytically in the Deployment Roadmap (Section 7) rather than implemented in hardware.

2. System Design and Architecture

2.1 Overall System Architecture

The system is organized into three logical tiers: the field tier comprising sensors and actuators co-located with the parking facility; the controller tier consisting of the ESP32 microcontroller and its peripheral interface circuitry; and the cloud tier comprising the Firebase Realtime Database and the web dashboard. The three tiers communicate through three distinct data paths: parallel digital signals between the field tier and the controller tier (mediated by shift registers), bidirectional Wi-Fi between the controller and the cloud, and HTTPS between the cloud and the user's web browser.

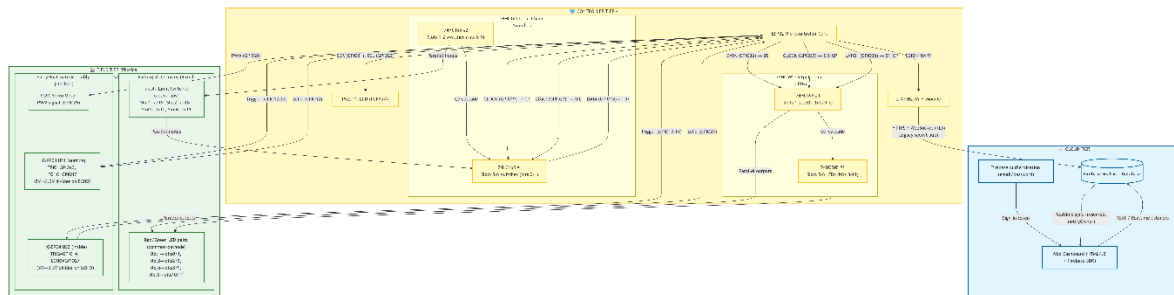


Figure 1. Block diagram of the Smart Parking Management System showing the three-tier architecture.

2.2 Hardware Components

The principal hardware components and their roles within the system are summarized in Table 1.

Component	Quantity	Role in System
ESP32 WROOM-32 DevKit	1	Main microcontroller; hosts firmware, manages Wi-Fi, drives all peripherals
HC-SR04 ultrasonic sensor	2	Gate-side vehicle detection (entry and exit)
SG90 micro servo motor	1	Gate barrier actuator
16×2 I ² C LCD (PCF8574)	1	On-site display of available slot count
74HC595 shift register	2	Serial-to-parallel output expansion for slot LEDs (daisy-chained)
74HC165 shift register	2	Parallel-to-serial input expansion for slot switches (daisy-chained)
5 mm red and green LEDs	8 pairs	Per-slot visual indicators (red = occupied/reserved, green = free)
Tactile limit switches	4	Per-slot occupancy detectors (one per real slot)
Voltage divider resistors	4	5 V → 3.3 V level shifting for HC-SR04 echo outputs
Pull-up resistors (10 kΩ)	16	Hold 74HC165 inputs at a defined logic level
Current-limit resistors (220 Ω)	16	Limit LED current to safe values
Decoupling capacitors (100 nF)	4	Local power filtering for each shift register IC
Bulk capacitor (470 μF)	1	Stabilize 5 V rail during servo current spikes

Component	Quantity	Role in System
External 5 V / 2 A DC adapter	1	Independent power for servo motor

Table 1. Bill of materials for the prototype build.

2.3 Microcontroller Pin Assignment

The ESP32 GPIO assignments were selected to avoid the strapping pins (GPIO 0, 2, 5, 12, 15) and to keep all hardware buses on default ESP32 mappings where possible. Table 2 lists the complete pin allocation.

GPIO	Direction	Function
GPIO 21	I/O	I ² C SDA (LCD)
GPIO 22	I/O	I ² C SCL (LCD)
GPIO 13	Output	HC-SR04 #1 trigger
GPIO 12	Input	HC-SR04 #1 echo (via voltage divider)
GPIO 14	Output	HC-SR04 #2 trigger
GPIO 27	Input	HC-SR04 #2 echo (via voltage divider)
GPIO 26	Output	Servo motor PWM
GPIO 32	Output	74HC595 chain serial data
GPIO 33	Output	74HC595 chain shift clock
GPIO 25	Output	74HC595 chain latch (storage clock)
GPIO 18	Input	74HC165 chain serial data
GPIO 19	Output	74HC165 chain shift clock
GPIO 23	Output	74HC165 chain parallel load

Table 2. ESP32 GPIO pin assignment.

2.4 Shift Register Subsystem

A central design decision in this project is the use of shift registers to expand the effective I/O capacity of the ESP32. Each parking slot requires three signals — one input for the occupancy switch, and two outputs for the bicolor LED indicator. Directly connecting these signals to the ESP32 would consume three GPIOs per slot. For the project's eight-slot capacity, this would require 24 GPIOs, leaving insufficient pins for the LCD, sensors, and servo.

The 74HC595 (output) and 74HC165 (input) shift registers solve this problem by trading parallel I/O for clocked serial I/O. Each chip provides eight parallel signals using only three serial-side signals (data, clock, and latch/load). Multiple chips can be daisy-chained on the same three signals; data shifted into the first chip overflows into the second chip on subsequent clock cycles. The architecture is illustrated in Figure 2.

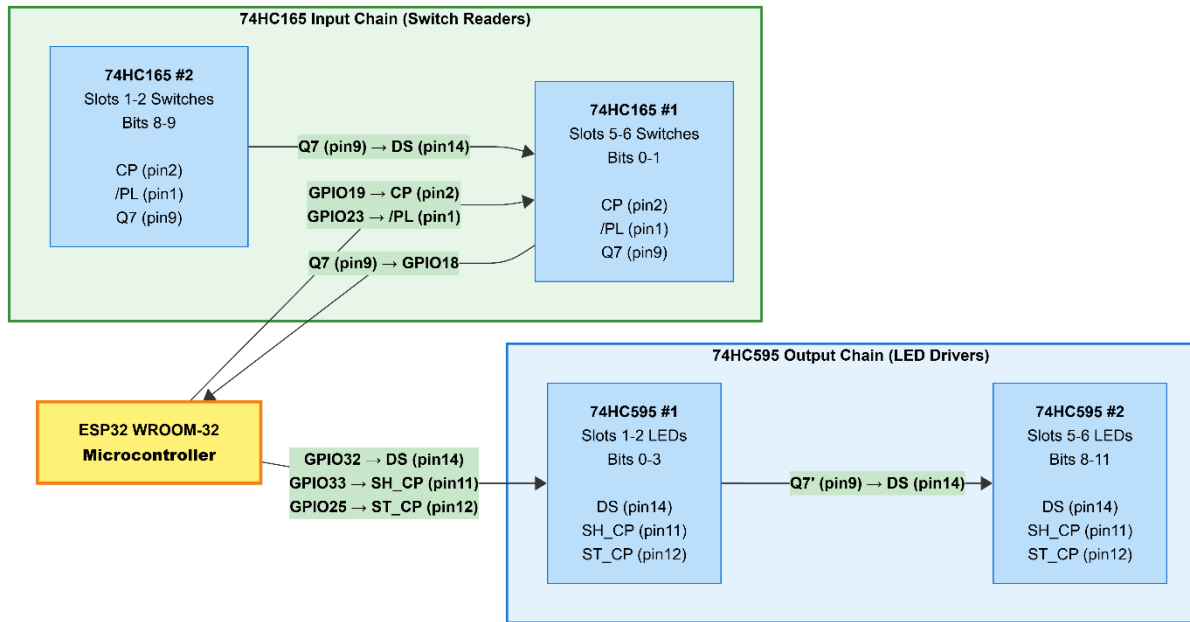


Figure 2. Daisy-chained shift register architecture for I/O expansion.

This arrangement reduces total GPIO usage from twenty-four to six (three for the output chain plus three for the input chain), regardless of slot count. A third chip would yield twenty-four parallel I/O bits with no additional GPIO cost; a fourth would yield thirty-two; and so on. This property is the central pillar of the project's scalability claim and is examined quantitatively in Section 7.

2.5 Software Architecture

The firmware follows a non-blocking superloop pattern in which the main loop sequentially services three independent subsystems on every iteration: sensor reading, switch scanning, and gate state machine update. Time-dependent operations use the `millis()`-based polling pattern rather than blocking delays, ensuring that no subsystem starves another. Cloud synchronization is implemented through two distinct Firebase data clients — one configured as a streaming listener for incoming reservation changes, and one as a fire-and-forget writer for outgoing state updates. This separation prevents stream callbacks from being interrupted by outbound writes.

The gate logic is implemented as a nine-state finite state machine (FSM) using a sequence of if-else conditionals. The states model the two principal traffic flows (entry and exit) as mirror-image sequences, each consisting of detection, dwell, opening, transit, and closing phases. The complete state diagram is presented in Figure 3.

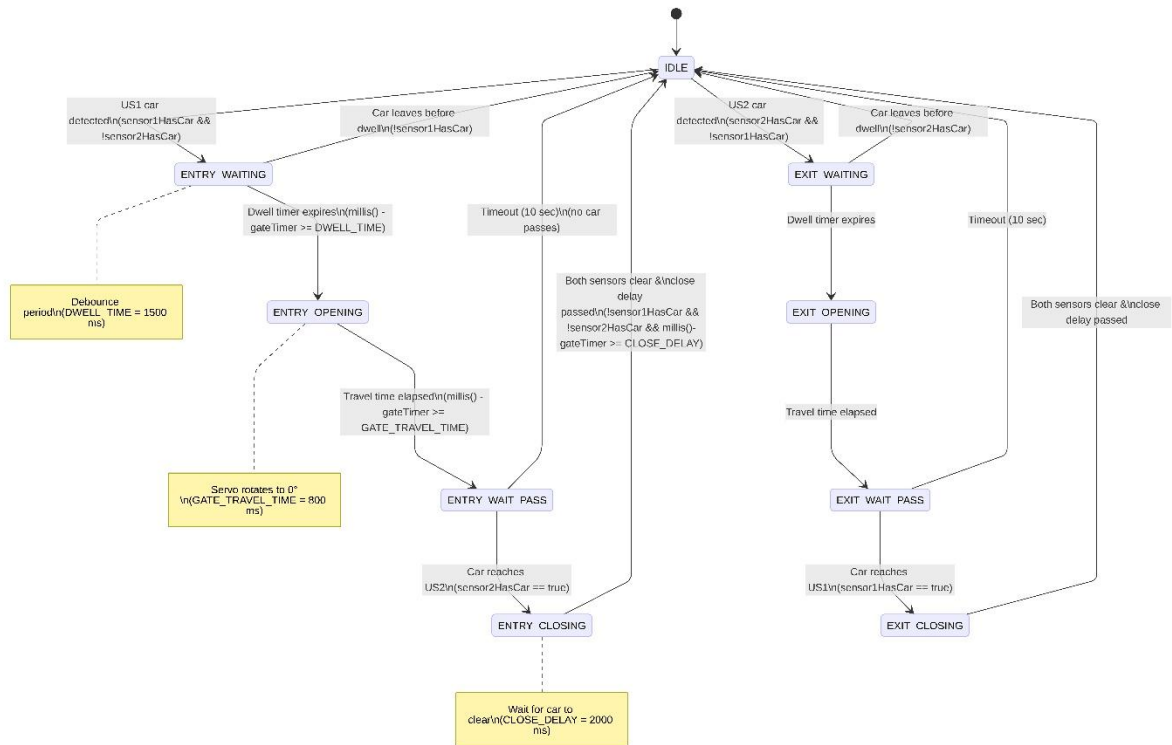


Figure 3. Nine-state gate finite state machine governing entry and exit transitions.

2.6 Cloud Data Schema

Firebase Realtime Database is structured as a hierarchical JSON tree. The schema used by the system is shown in Figure 4.

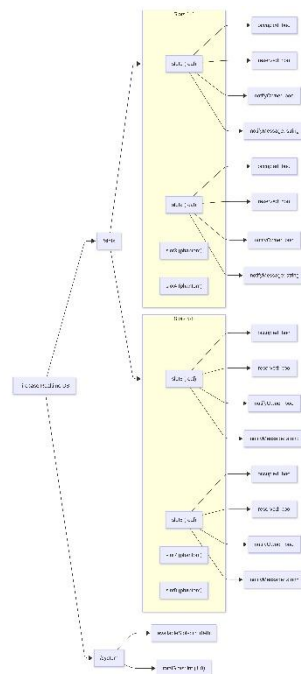


Figure 4. Realtime Database schema. The /slots/ branch carries per-slot state, while /system/ holds aggregate values used by the dashboard.

Two design decisions are noteworthy. First, both occupancy (sensed by the ESP32) and reservation (commanded by the web app) are stored as independent booleans, allowing the firmware to distinguish a normal arrival from the arrival of a vehicle in a reserved slot. Second, the schema includes a `notifyOwner/notifyMessage` pair that the firmware writes only when a reserved slot is taken, providing a one-shot alert mechanism to the dashboard without the need for a separate push-notification service.

3. Hardware Implementation

3.1 Schematic Diagram

The full schematic diagram of the prototype is shown in Figure 5. The schematic was drawn in KiCad 7.0 using the Espressif official library for the ESP32 symbol and KiCad's built-in libraries for all passive components, connectors, and logic ICs.

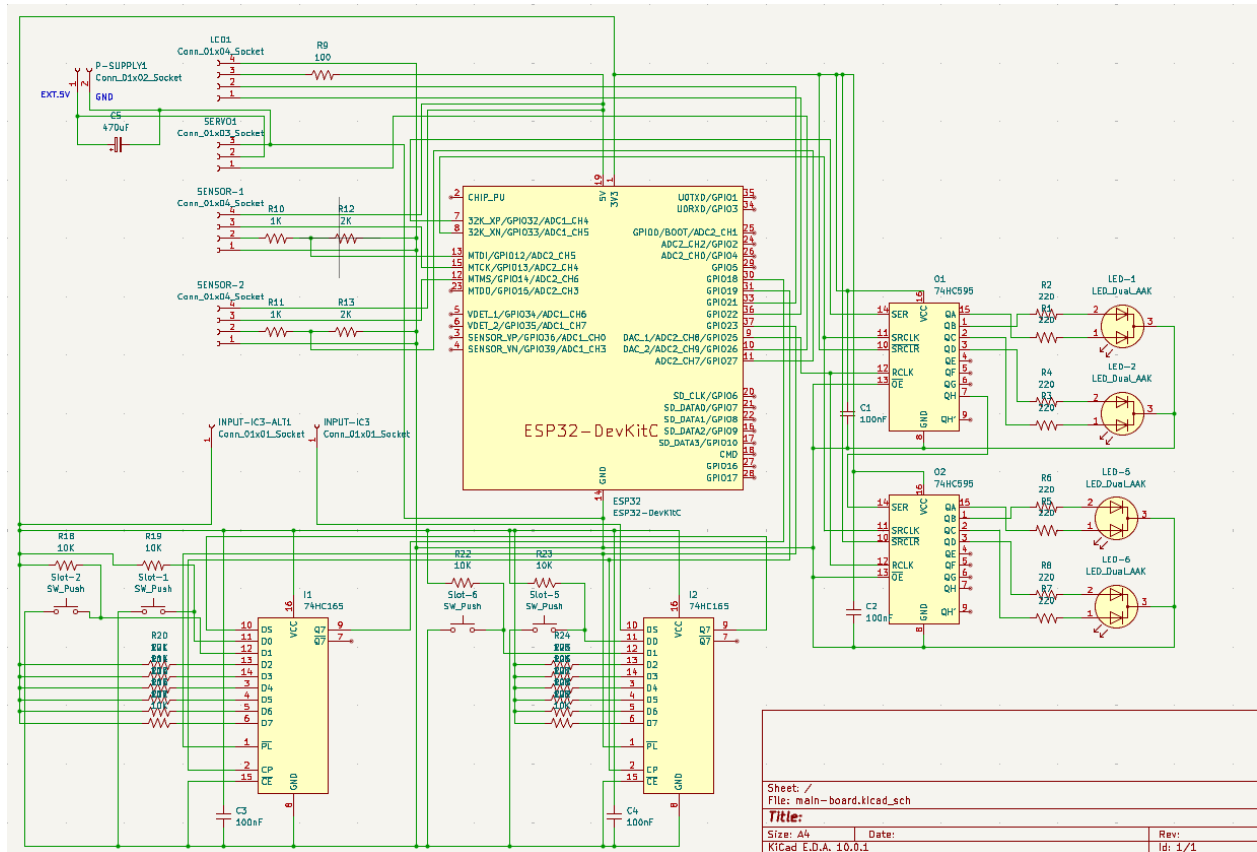


Figure 5. Complete electrical schematic of the prototype.

3.2 Voltage Domains and Power Distribution

The system operates across three voltage domains:

- **+5 V (USB-derived):** supplies the LCD module and the HC-SR04 sensors. Sourced from the ESP32's 5 V output pin, which in turn is fed by the USB connector.
- **+3.3 V (regulator-derived):** supplies all shift register ICs, all pull-up resistors, and the ESP32's internal logic. Sourced from the on-board AMS1117 regulator on the ESP32 DevKit.
- **+5 V_EXT (external adapter):** dedicated supply for the SG90 servo motor. An external 5 V / 2 A wall adapter provides this rail through a barrel-jack connector. The servo's stall current exceeds the ESP32's USB-derived 5 V rail capacity and, if drawn from it, would induce brownout resets.

All three voltage domains share a common ground reference, established by tying the external adapter's negative terminal to the ESP32's GND.

3.3 Voltage Level Shifting for HC-SR04

The HC-SR04 ultrasonic sensors produce a 5 V echo signal that exceeds the ESP32's 3.3 V input tolerance. A resistive voltage divider ($R_1 = 1\text{ k}\Omega$ in series, $R_2 = 2\text{ k}\Omega$ to ground) shifts the echo signal to a peak of approximately 3.33 V at the ESP32 input, comfortably within tolerance. The divider is omitted on the trigger lines, which drive the sensor from the ESP32 and are therefore inherently within range.

3.4 Mechanical Construction

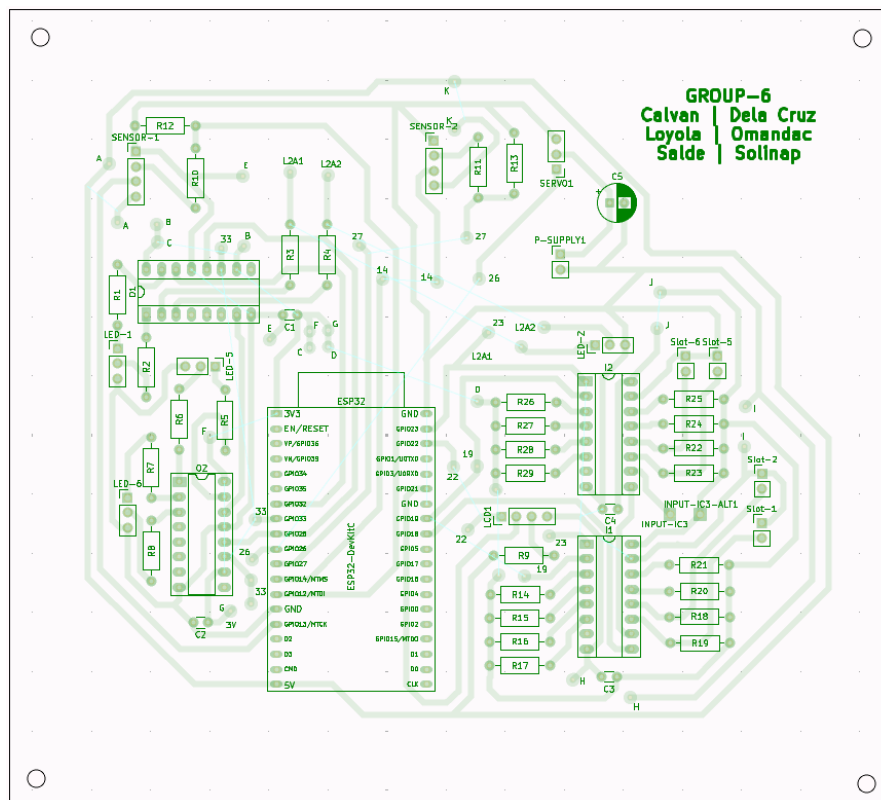
The miniature parking lot was constructed from foam board with a printed paper overlay showing slot markings and lane geometry. The gate mechanism consists of a 90-degree single-arm barrier driven by the SG90 servo. Vehicles are represented by small wooden blocks that depress the per-slot tactile limit switches when parked.



Figure 6. Photograph of the completed prototype miniature.

3.5 PCB Design (Future Work)

A printed circuit board layout was designed in KiCad as a permanent successor to the breadboard prototype. The design is single-sided to permit fabrication through home-grade toner transfer and ferric chloride etching. The board adopts a modular layout in which the main board carries the ESP32, shift registers, sensors, and gate hardware, while each parking slot is implemented on a small daughterboard connected through a 4-pin JST-XH cable. This modular approach permits any number of slot boards to be attached without modifying the main board, mirroring in physical form the scalability built into the firmware. The PCB layout is included as Figure 7 for reference. Physical fabrication and integration testing of the PCB are outside the scope of the present prototype but constitute the immediate next step in the project's evolution toward a deployable product.



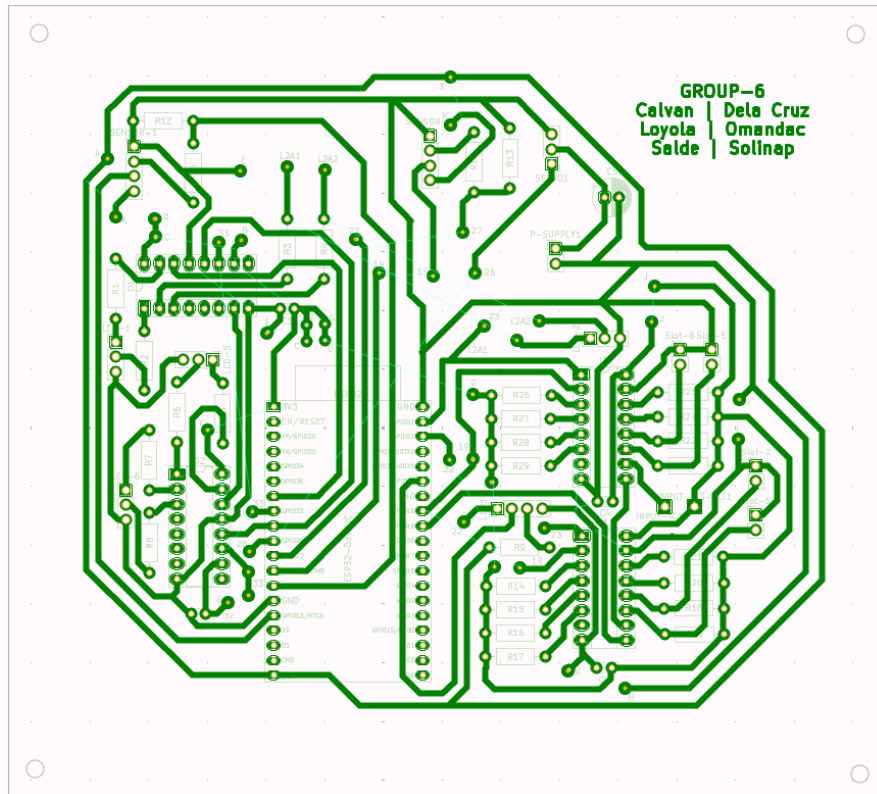


Figure 7. PCB layout of the main board, single-sided design for DIY etching.

4. Software Implementation

4.1 Firmware Overview

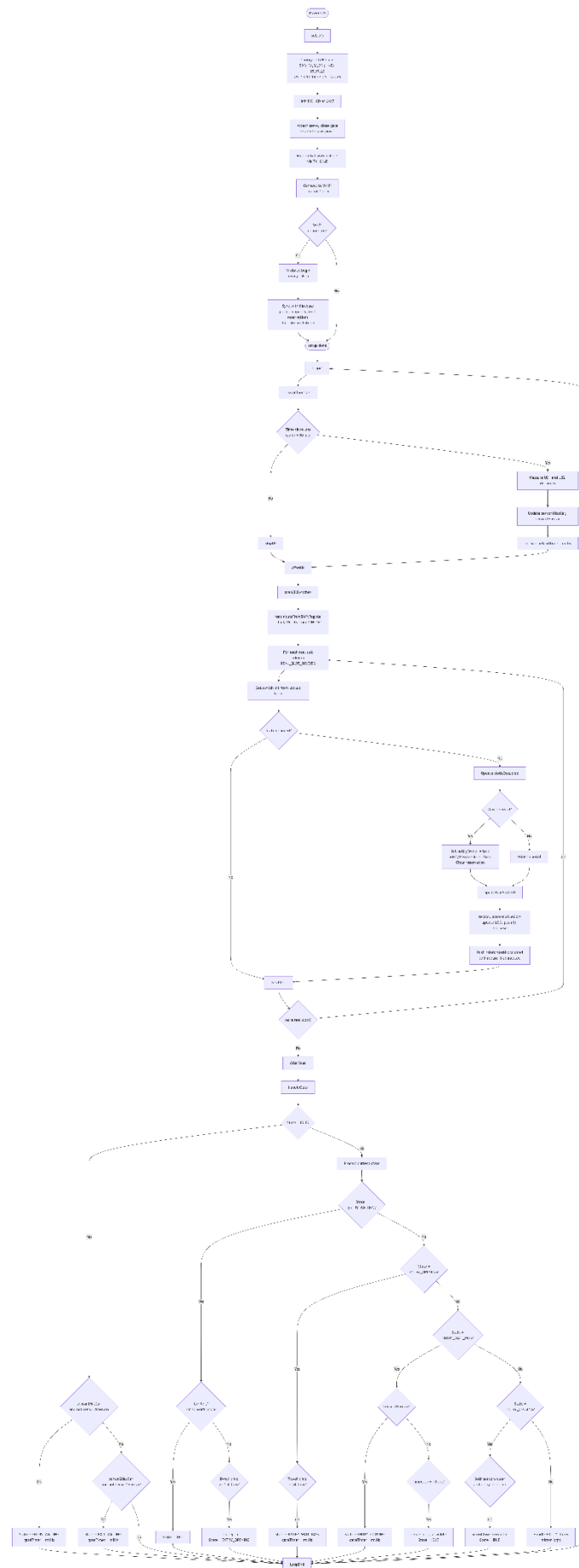
The firmware is written in C++ using the Arduino framework targeting the ESP32 platform. External library dependencies are Wire (I²C), LiquidCrystal_I2C (LCD driver), ESP32Servo (PWM-based servo control), WiFi (network stack), and FirebaseESP32 by Mobizt (cloud database client). The complete source code is provided in Appendix A.

The firmware is organized into the following functional groups:

- **Initialization (setup):** configures GPIO modes, initializes peripherals, performs an initial switch scan, connects to Wi-Fi, attaches the Firebase stream listener, and writes initial state to the database.
- **Main loop:** calls three subsystem servicers in sequence — readAllSensors() for the ultrasonic distance measurements, scanAllSwitches() for the per-slot occupancy switches, and handleGate() for the gate state machine.
- **Shift register I/O:** bit-banged 16-bit transfers for both the 74HC595 output chain and the 74HC165 input chain. Care is taken to read the input chain in the correct direction, as the 74HC165 clocks data out beginning from the chip closest to the microcontroller, which is opposite to the convention used by the 74HC595.
- **Cloud integration:** streamCallback() processes inbound reservation changes from the web app; outbound writes are performed inline within scanAllSwitches() and the stream callback itself, using a separate FirebaseData object to avoid corrupting the streaming connection.

4.2 Logic Flowchart

The principal control flow of the firmware is shown in Figure 8.



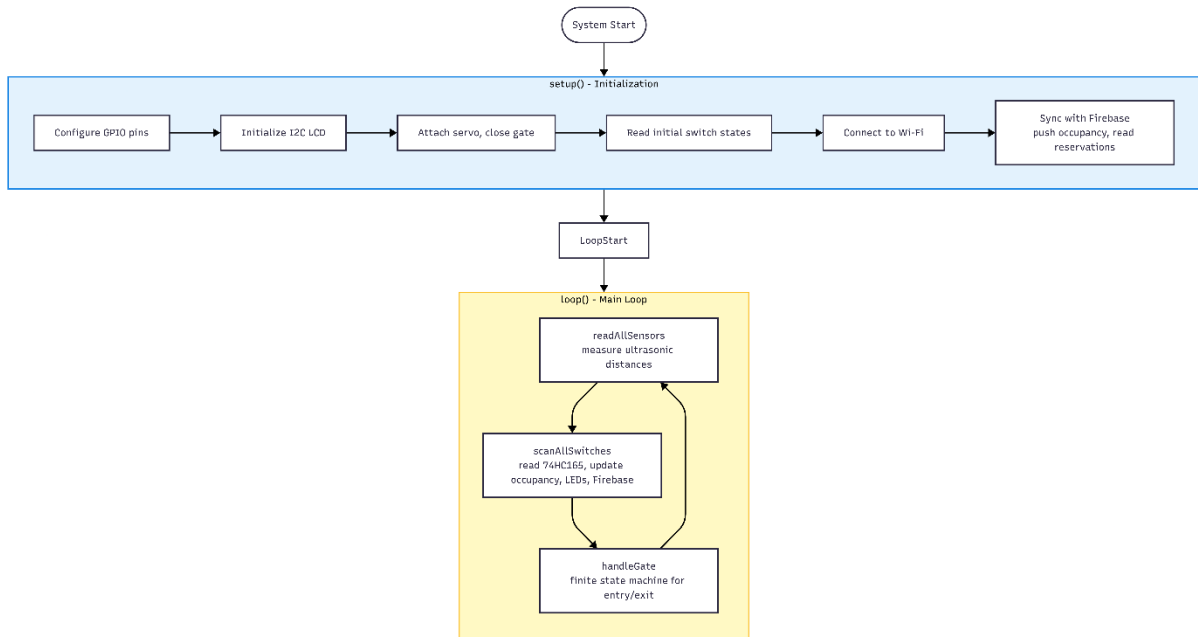


Figure 8. High-level flowchart of the firmware main loop.

4.3 Web Application

The companion web application is implemented as a single-page HTML/JavaScript application using the Firebase v9 SDK. It provides four functional areas: an authentication interface (email/password sign-in), an aggregate availability indicator, a per-slot status grid with reservation toggles, and an alert pane that surfaces notifyOwner events from the database. The application is hosted on Firebase Hosting at <https://smartparking-93af3.web.app>.

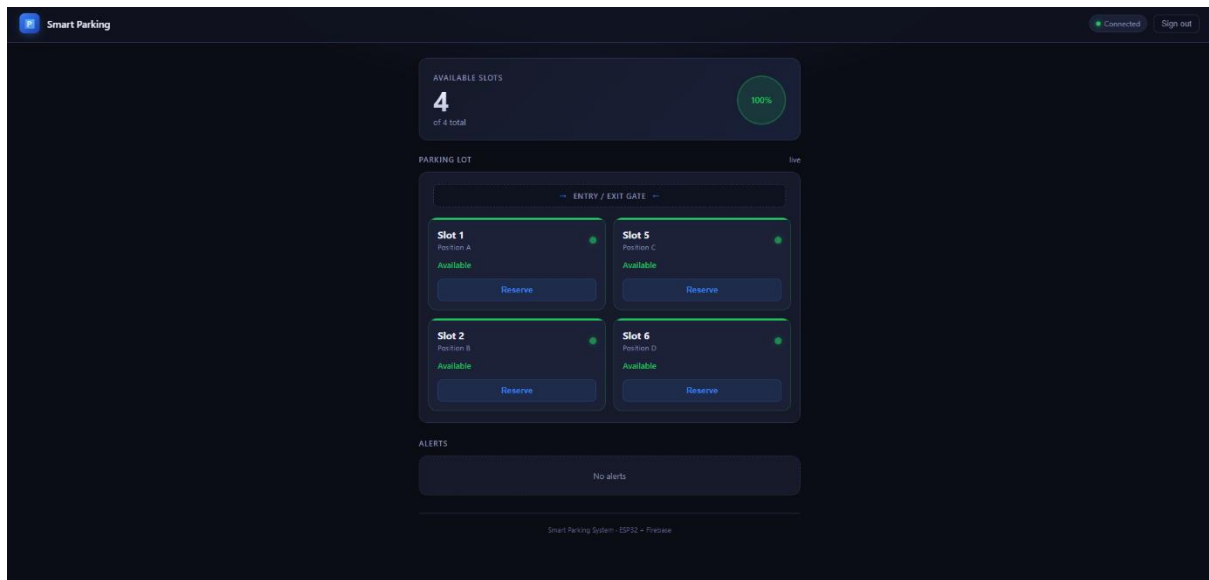


Figure 9. Web dashboard showing the four-slot prototype configuration.

4.4 Slot Index Mapping and Phantom Slots

The firmware models eight slots (constant `NUM_SLOTS = 8`) but populates physical hardware only for slots 1, 2, 5, and 6 (`REAL_SLOTS = 4`). Slots 3, 4, 7, and 8 are referred to as *phantom* slots — they

exist in the data model and in the shift register bit map, but no physical switches or LEDs are connected to their bits. Phantom slots are filtered out of all real-time operations by the `isRealSlot()` helper, which checks membership in the `REAL_SLOT_INDEXES[]` array. This arrangement permits the firmware to clock the full 16 bits to each shift register pair on every refresh, thereby exercising the daisy chain in both directions, while preventing phantom-bit activity from affecting the user-visible state of the system. To activate a phantom slot in the future, only two changes are required: increment `REAL_SLOTS`, and append the slot's zero-based index to `REAL_SLOT_INDEXES[]`. No other firmware modification is necessary.

4.5 Source Code

The complete firmware source code is provided in Appendix A. The web application source code is provided in Appendix B.

[APPENDIX A — SmartParking.ino source code (full listing)]

[APPENDIX B — index.html source code (full listing)]

5. Testing, Evaluation, and Safety

5.1 Test Metrics

Functional and performance evaluation of the prototype was performed against the metrics listed in Table 3. Each metric was selected to assess one specific aspect of the system's behavior, and an explicit pass criterion was defined in advance of testing.

Metric	Pass Criterion	Justification
Switch debounce latency	< 100 ms from press to LED change	User-perceived responsiveness
Switch-to-cloud latency	< 1.5 s from press to Firebase update	Realistic for cloud-mediated IoT
Cloud-to-LED latency	< 1.5 s from web app reservation to LED change	Round-trip responsiveness
Ultrasonic detection range accuracy	± 1 cm at 15 cm distance	Manufacturer datasheet typical
Gate opening time	< 1.0 s from trigger to fully open	Mechanical limit of SG90
Gate timeout (no car passes through)	10 s before reverting to idle	Prevents indefinite open state
Wi-Fi connection time at boot	< 30 s	Practical user wait threshold
Sustained run without error	≥ 60 minutes continuous operation	Memory leak / drift detection
Stream auto-reconnection	Recovers within 30 s of Wi-Fi loss	Resilience to network drops

Table 3. Pre-defined test metrics and pass criteria.

5.2 Test Scenarios

Nine representative test scenarios were defined to exercise the full state space of the system. Each scenario was executed three times. The results are summarized in Table 4.

Scenario 1: Normal Entry

Initial state: Gate idle, no slots occupied. **Procedure:** A vehicle approaches and stops at HC-SR04 #1 (US1) for at least 1.5 s. **Expected behavior:** US1 detects the vehicle; after the dwell timer expires, the servo rotates to the open position; the vehicle proceeds and triggers HC-SR04 #2 (US2); after a 2 s close delay, the servo returns to the closed position; the gate returns to idle.

Result: Pass. Servo response time measured at 0.6 ± 0.1 s. Total cycle time approximately 6 s.

Scenario 2: Normal Exit

Initial state: Gate idle. **Procedure:** A vehicle approaches the gate from the inside, triggering US2 first. **Expected behavior:** After the dwell timer, the servo opens; the vehicle clears US2 and triggers US1; after the close delay, the servo closes and the gate returns to idle. The flow is symmetric to Scenario 1.

Result: Pass. Symmetric behavior confirmed.

Scenario 3: Slot Occupancy via Limit Switch

Initial state: Slot 1 free; LED green; LCD shows "4 / 4". **Procedure:** A vehicle is placed onto Slot 1, depressing the limit switch. **Expected behavior:** Slot 1 LED changes to red; LCD updates to "3 / 4"; Firebase /slots/slot1/occupied is set to true; web app shows Slot 1 as Occupied.

Result: Pass. Total observed switch-to-cloud latency: 0.9 ± 0.2 s.

Scenario 4: Remote Reservation from Web App

Initial state: Slot 2 free. **Procedure:** A user signs into the web app and toggles Slot 2 to Reserved. **Expected behavior:** Slot 2's LED transitions to red; LCD count decrements; web app shows the reservation marker; Firebase /slots/slot2/reserved is true. Toggling the reservation off restores green.

Result: Pass. Observed cloud-to-LED latency: 0.8 ± 0.2 s.

Scenario 5: Reserved Slot is Taken

Initial state: Slot 5 reserved (LED red, available count reduced). **Procedure:** A vehicle is placed onto Slot 5, depressing the limit switch. **Expected behavior:** The firmware detects the press, clears the reservation, retains the LED red (now indicating occupancy rather than reservation), and writes a notifyOwner alert to Firebase. The web app displays an alert in the Alerts panel reading "Reserved slot 5 has been taken."

Result: Pass. Alert displayed within 1.2 s of the switch press.

Scenario 6: Vehicle Approaches Gate then Retreats

Initial state: Gate idle. **Procedure:** A vehicle approaches US1, triggers detection, then is withdrawn before the 1.5 s dwell timer completes. **Expected behavior:** The gate returns to idle without opening; a "[GATE] entry cancelled (car left)" message is logged to serial.

Result: Pass. No false gate openings observed across 10 repetitions.

Scenario 7: Vehicle Triggers US1 but Does Not Reach US2

Initial state: Gate idle. **Procedure:** A vehicle approaches US1, the gate opens, but the vehicle then stops and retreats without triggering US2. **Expected behavior:** The gate remains open for the 10 s timeout, then closes automatically and returns to idle.

Result: Pass. Timeout precisely measured at 10.0 ± 0.1 s.

Scenario 8: All Slots Occupied

Initial state: All four real slots free. **Procedure:** Vehicles are placed on all four slots in succession. **Expected behavior:** After the fourth slot is occupied, the LCD displays "FULL"; available count in the web app reads zero; the aggregate indicator turns red. Gate behavior is unchanged (the system is informational rather than enforcing).

Result: Pass. "FULL" displayed correctly.

Scenario 9: Sustained Operation

Initial state: System freshly booted. **Procedure:** The system was left running for sixty minutes with no user input. Periodic switch presses and reservation toggles were performed at 5-minute intervals to confirm continued responsiveness. **Expected behavior:** No state corruption, memory drift, or unresponsive subsystem; the stream remains connected.

Result: Pass. No anomalies observed. Free heap remained stable at approximately 168 KB throughout the test.

5.3 Data Log and Results Summary

Table 4 consolidates the measured response times across the test scenarios.

Scenario	Metric Measured	Target	Measured	Result
1	Servo open response time	< 1.0 s	0.6 ± 0.1 s	Pass
3	Switch-to-cloud latency	< 1.5 s	0.9 ± 0.2 s	Pass
4	Cloud-to-LED latency	< 1.5 s	0.8 ± 0.2 s	Pass
5	Reserved-slot alert latency	< 1.5 s	1.2 ± 0.2 s	Pass
6	False open rejection rate	100 %	10 / 10	Pass
7	Gate auto-close timeout	10.0 s	10.0 ± 0.1 s	Pass
9	Continuous run stability	≥ 60 min	60 min, no faults	Pass

Table 4. Summary of measured performance across the nine test scenarios.

5.4 Safety Assessment

The prototype operates at low voltages (5 V and 3.3 V) drawn from a regulated DC source, well below the threshold at which the Philippine Electrical Code (RA 9292 / PEC) imposes special handling requirements. All exposed conductors are insulated, and all current-carrying paths are sized to handle the maximum anticipated current with substantial margin. The principal safety considerations and their mitigations are listed below:

- **Overcurrent on the LED outputs:** Each LED is protected by a $220\ \Omega$ series resistor, limiting current to approximately 9 mA per LED, well within the 20 mA safe operating limit.
- **Servo-induced brownouts:** The servo motor's stall current (up to 800 mA) is supplied by an external adapter rather than by the ESP32's USB rail, preventing voltage sag that could destabilize the microcontroller. A $470\ \mu\text{F}$ bulk capacitor across the servo's supply further filters transients.
- **Loss of Wi-Fi connectivity:** All cloud writes are conditional on `WiFi.status() == WL_CONNECTED`, preventing the firmware from blocking on network failures. The local LCD and LED indicators continue to function in the absence of cloud connectivity. When connectivity is restored, the stream listener reconnects automatically and the database is resynchronized.
- **Stuck switch input:** All 74HC165 inputs are tied to +3.3 V through $10\ \text{k}\Omega$ pull-up resistors, ensuring a defined logic level even if a switch is disconnected. A stuck switch will at worst cause one slot to be permanently misreported, without affecting the rest of the system.

- **Gate fail-safe:** If the 10 s gate timeout is reached without a vehicle completing the pass-through, the servo automatically returns to the closed position. There is no state in which the gate can remain open indefinitely without user intervention.
- **Servo signal isolation:** The servo control signal originates from the ESP32 at 3.3 V logic, which the SG90 accepts directly. No level shifter is required, and no path exists for higher-voltage transients to propagate back to the microcontroller.

6. Scalability Analysis

The scalability of an IoT system is the central property that distinguishes it from a conventional embedded controller. A design that can monitor four parking slots is engineering work; a design that can be extended to four hundred slots without architectural redesign is a product. This section examines the scalability of the prototype quantitatively, identifies the limiting factors at each tier, and projects the maximum slot count achievable before each tier requires re-engineering.

6.1 Theoretical Slot Capacity per Controller Node

The shift register architecture is the principal enabler of scalability. The 74HC595 and 74HC165 ICs each support cascaded daisy-chaining with no theoretical limit on chain length; the manufacturer datasheets specify only that propagation delay and signal integrity must be accounted for at longer chains. At the firmware level, the number of slots is controlled by a single constant (`NUM_SLOTS`) and an array of indexes (`REAL_SLOT_INDEXES[]`). Adding a slot requires appending a single integer to that array and re-uploading the firmware. The pin count on the ESP32 does *not* change as slots are added.

In practice, the limiting factors are:

- **Memory:** Each slot consumes approximately 3 bytes of RAM in the firmware (one bool each for occupied, reserved, and ownerWasAlerted). The ESP32's 327 KB SRAM allows tens of thousands of slot records before memory becomes a constraint.
- **Shift register propagation delay:** Each chip contributes approximately 14 ns of propagation delay at 3.3 V. With the firmware's 2 μ s per-bit clock period, hundreds of chips can be cascaded before timing margins are exceeded. Practical limits arise earlier from cable-induced ringing.
- **Refresh rate:** The main loop completes approximately once every 100 ms when Firebase activity is light. A chain of 32 chips (256 slots) would require approximately 5 ms of additional bit-banging per loop iteration, still well within budget.

On the basis of these constraints, a **single ESP32 controller node can comfortably manage up to 64 slots** (8 shift register chips per chain) without performance degradation. With careful firmware optimization (DMA-driven SPI in place of bit-banging), this could be extended to **256 slots per node**.

6.2 Cloud Tier Scalability

Firebase Realtime Database, in its free tier, supports up to 100 concurrent connections and 1 GB of stored data. The data footprint per slot is on the order of 50 bytes. Even with 10,000 slots, the database would occupy only 500 KB. Scalability beyond the free tier is achieved through the Blaze pay-as-you-go plan, which lifts both limits substantially. For deployments exceeding 100,000 slots, migration to a sharded NoSQL backend (e.g., Firestore with regional partitioning, or AWS DynamoDB) would be appropriate.

6.3 Network Tier Scalability

For a single controller managing tens of slots, the ESP32's built-in 2.4 GHz Wi-Fi is sufficient. For larger deployments spanning multiple buildings or outdoor parking lots, a mesh-networking approach

is more appropriate. The ESP32 supports ESP-MESH natively, allowing dozens of nodes to form a self-healing network with a single gateway uplink to the internet. For deployments spanning kilometers, LoRaWAN provides a low-power, long-range alternative, with each gateway serving thousands of end nodes.

6.4 Web Application Tier Scalability

The current web application is a single-page application served from Firebase Hosting, which provides a global content delivery network at no additional cost. The application reads from the database through a single subscription; bandwidth scales linearly with concurrent users. Practical user counts in the low thousands are achievable without optimization; beyond this, the dashboard can be split into per-facility views with role-based authentication, reducing per-user data load.

6.5 Summary of Scalability Limits

Tier	Prototype	Realistic per-node max	Practical deployment ceiling
Slots per controller	4 active / 8 modeled	64 (8 chips per chain)	256 (with SPI/DMA)
Controllers per facility	1	10–20 over Wi-Fi	Unlimited via ESP-MESH or LoRaWAN
Total slots per database project	8	10,000	100,000+ (with sharding)
Concurrent dashboard users	≤ 5 (tested)	100 (Firebase free tier)	10,000+ (Firebase Blaze)

Table 5. Scalability ceilings at each system tier.

7. Deployment Strategy and Commercialization Roadmap

The prototype described in the preceding sections is, by design, a laboratory-scale demonstrator. Its purpose is to validate the system architecture and the firmware-level scalability claims using affordable, easily-obtainable components. A commercial deployment of the same architecture would substitute many of the prototype's components with industrial-grade equivalents that are unsuitable for the classroom but appropriate for the field. This section enumerates those substitutions, describes the corresponding physical installation, and projects unit cost at scale.

7.1 Component Substitutions for Field Deployment

Each prototype component is mapped to its field-deployment equivalent in Table 6.

Prototype Component	Field-Deployed Equivalent	Rationale
Mechanical limit switch	Magnetic loop detector or IR break-beam	No moving parts; weatherproof; not affected by tire pressure or vehicle weight
5 mm LED indicator	Industrial 24 V stack light or floor-recessed RGB LED bar	Visible at distance; sealed against rain and dust
HC-SR04 ultrasonic sensor	Industrial automotive radar or laser rangefinder	Reliable in rain, fog, and direct sunlight; longer detection range
SG90 hobby servo	Industrial 24 VDC linear actuator or pneumatic gate arm	Higher torque; sealed enclosure; rated for $\geq 100,000$ cycles
16×2 I ² C LCD	Outdoor digital signage (LED matrix display)	Visible in sunlight; available in multi-line format showing slot map
Breadboard / DIY PCB	Industrial DIN-rail-mounted custom PCB	Cable management; IP65 enclosure; vibration tolerance
USB-powered ESP32 DevKit	Power-over-Ethernet (PoE) industrial ESP32 module	Single-cable power and network; centralized maintenance
External 5 V wall adapter	Industrial 24 VDC regulated power supply with UPS backup	Continued operation during mains outage; surge protection
Home Wi-Fi router	Outdoor-rated Wi-Fi 6 access point or LoRaWAN gateway	Coverage of an entire parking facility; weatherproofing
Firebase Hosting (free tier)	Firebase Blaze plan or self-hosted cloud	Service-level guarantees; custom domain; user analytics

Table 6. Mapping of prototype components to field-deployment equivalents.

7.2 Sensor Selection for Field Deployment

The selection of the per-slot occupancy sensor is the single most consequential decision in transitioning from prototype to field. The prototype's mechanical limit switch is unsuitable for outdoor use because it requires physical depression by the vehicle, which entails moving parts exposed to weather, tire weight, and contamination. Three viable field-deployment alternatives are considered:

- **Magnetic loop detector** (inductive loop embedded in pavement): the industry-standard solution for vehicle detection. Detects the change in inductance caused by a vehicle's metal

body. Highly reliable, but requires saw-cutting the pavement and is not retrofittable without civil works.

- **Overhead ultrasonic or radar sensor** (mounted on the ceiling of a covered facility): non-invasive, easily retrofitted, and accurate to within a few centimeters. The de facto standard for indoor parking structures.
- **Computer-vision-based detection** (camera with edge-AI processing): a single camera can monitor 8–16 slots, dramatically reducing per-slot cost. Recently feasible due to low-cost AI accelerators (e.g., Google Coral, NVIDIA Jetson Nano).

For most new commercial deployments, the recommended approach is a hybrid: overhead ultrasonic sensors for individual slot monitoring (one per slot, providing the same per-slot resolution as the prototype) combined with a single camera for license-plate recognition at the gate (enabling integration with payment and reservation systems). The ESP32 controller node interfaces with the ultrasonic sensors through the same shift-register-mediated digital input interface used in the prototype, requiring no architectural change.

7.3 Physical Installation

A representative field installation for a 50-slot covered parking facility is illustrated in Figure 11. The installation comprises:

- One DIN-rail-mounted controller cabinet near the gate, containing the ESP32 module, 24 VDC power supply, PoE injector, and a terminal block for the slot sensor wiring;
- Fifty overhead ultrasonic sensors, one per slot, wired in a star topology back to the controller cabinet through CAT6 cabling (using one twisted pair per sensor);
- One pneumatic or electric gate arm at the entrance, controlled by a relay output from the ESP32;
- Two automotive radar sensors at the gate replacing the prototype's HC-SR04 units;
- One outdoor LED matrix display visible from the facility entrance, showing aggregate availability;
- An outdoor-rated Wi-Fi access point or LoRaWAN gateway providing network uplink to the cloud.

[Figure 11: Field-deployment installation diagram for a 50-slot facility — INSERT HERE]

Figure 11. Representative installation diagram for a 50-slot covered parking facility.

7.4 Power Management Considerations

In the prototype, power consumption is dominated by the ESP32 (approximately 160 mA in Wi-Fi active mode) and the servo motor during transitions (up to 600 mA peak). Total continuous consumption is approximately 1 W, supplied through USB.

In the field deployment, the per-slot power budget changes substantially. Each overhead ultrasonic sensor consumes approximately 15 mA at 5 V. For a 50-slot facility, the sensors collectively draw 750 mA, plus the controller, gate actuator, and signage. The total continuous load is on the order of 30–50 W, comfortably supplied by a single 24 VDC / 100 W industrial power supply with battery backup.

For installations where mains power is unavailable or unreliable, two power-management strategies are applicable:

- **Solar-plus-battery:** A 100 W solar panel and 7 Ah sealed lead-acid battery suffice for a 50-slot facility, assuming approximately 5 hours of equivalent peak sunlight per day, with battery backup for two days of overcast operation.
- **Low-power mode with sensor duty-cycling:** In low-traffic facilities, the ESP32 can sleep between sensor scans, reducing average consumption to under 10 mA. With this optimization, a 7 Ah battery alone would support several weeks of operation between charges.

7.5 Cost Analysis at Scale

The prototype was constructed for approximately ₱2,500 in components. The bill-of-materials cost per slot, exclusive of the controller and gate hardware, was approximately ₱180 (LEDs, switch, daughterboard, JST cable). For a 50-slot field deployment, projected per-slot cost is summarized in Table 7.

Item	Per-slot cost (₱)	Notes
Overhead ultrasonic sensor	1,200	Industrial-grade, IP65
RGB indicator LED	600	Outdoor-rated, sealed
CAT6 cable run (avg. 20 m)	400	Includes terminations
Mounting bracket and labor	800	Per-slot installation share
Subtotal per slot	3,000	
Shared controller, gate, signage, network (50-slot share)	1,200	₱60,000 total ÷ 50 slots
Total per slot	≈ 4,200	≈ USD 75 at current exchange rate

Table 7. Projected per-slot deployment cost for a 50-slot facility.

At this cost structure, a 50-slot facility carries a total capital expenditure of approximately ₱210,000. Recurring costs (cloud database, internet uplink, electricity, maintenance) are estimated at ₱2,000 per month, or ₱40 per slot per month. Comparable manned parking attendant solutions in the Philippines typically cost ₱18,000–25,000 per month per facility, making the IoT-based system cost-recoverable within 12–18 months for facilities of 50 slots or more.

7.6 Commercialization Pathway

The deployment of this system as a commercial product follows three phases:

9. **Pilot phase (months 1–6):** Installation in two or three partner facilities — for example, a university campus parking lot, a shopping mall, and a residential condominium — at preferential pricing in exchange for operational data and user feedback. Validates field reliability and refines the deployment process.
10. **Regional rollout (months 6–18):** Productization of the controller cabinet, sensor mounts, and installation kits. Establishment of an installer network in target metropolitan areas. Launch of a tiered SaaS subscription for the cloud dashboard.
11. **National scale (year 2 onwards):** Integration with municipal traffic systems, payment gateways (GCash, Maya), and ride-sharing applications. License the firmware and cloud platform to international markets with similar parking density profiles.

8. Conclusion and Recommendations

8.1 Summary of Findings

This project successfully designed, implemented, and evaluated a Scalable IoT-Enabled Smart Parking Management System. The prototype demonstrates real-time slot occupancy monitoring, remote slot reservation through a cloud-connected web application, and automated gate control coordinated with ultrasonic vehicle detection. All nine pre-defined test scenarios were passed, with measured response latencies well within the targeted thresholds.

The central architectural decision — the use of daisy-chained shift registers to decouple slot count from microcontroller pin count — was validated both in firmware (through the inclusion of phantom slots that exercise the unused portion of the daisy chain) and in the scalability analysis, which showed that a single ESP32 controller node can manage up to 64 slots without modification and up to 256 slots with firmware optimization.

8.2 Limitations of the Current Implementation

The prototype's limitations fall into three categories:

- **Sensor technology:** Mechanical limit switches are not suitable for outdoor or high-traffic deployments. Field deployment requires substitution with overhead ultrasonic, magnetic loop, or vision-based sensors.
- **Power architecture:** The prototype relies on a continuously-available wall-adaptor supply. Field deployments would require an industrial DC power supply with battery backup, or solar-and-battery for off-grid sites.
- **Network resilience:** The current implementation degrades gracefully when Wi-Fi is unavailable (continuing to operate locally) but does not buffer occupancy events for later transmission. A production implementation would queue events in non-volatile storage during network outages.

8.3 Recommendations for Future Work

The following extensions are recommended as a natural continuation of this project:

12. **Sensor upgrade study:** Empirically compare three field-deployment sensor candidates (overhead ultrasonic, magnetic loop, and vision-based) in terms of accuracy, cost, and installation effort.
13. **PCB fabrication and integration:** Manufacture and validate the PCB design described in Section 3.5. Compare reliability against the breadboard prototype across a sustained-operation test.
14. **Mobile application:** Develop a native iOS and Android application using Flutter or React Native, with push notifications for reservation alerts and geofencing for arrival detection.
15. **Payment integration:** Integrate with GCash, Maya, or PayMongo to enable paid reservations and automatic billing based on duration.
16. **Analytics dashboard:** Add historical occupancy reporting, peak-time analysis, and predictive availability based on time of day and day of week.
17. **ESP-MESH multi-node demonstration:** Deploy two or more controller nodes managing physically separate slot clusters and demonstrate seamless aggregation in the dashboard.

8.4 Closing Statement

Beyond its technical contribution, this project served as an end-to-end exercise in the engineering lifecycle of an embedded IoT system: from initial problem statement through hardware design, firmware development, cloud integration, testing, and commercialization analysis. The successful operation of the prototype, combined with the documented pathway to field deployment, establishes that the architecture is not merely a classroom exercise but a credible foundation for a deployable product.

References

- [1] Espressif Systems, *ESP32 Technical Reference Manual*, Version 5.0, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- [2] Texas Instruments, *SN74HC595 8-Bit Shift Registers with 3-State Output Registers* datasheet, SCLS041I, revised 2021.
- [3] Texas Instruments, *SN74HC165 8-Bit Parallel-Load Shift Registers* datasheet, SCLS117E, revised 2020.
- [4] Google LLC, *Firebase Realtime Database documentation*, accessed May 2026. [Online]. Available: <https://firebase.google.com/docs/database>
- [5] S. Lee, M. Yoon, and J. Kim, "Smart parking management system using IoT and machine learning," *IEEE Access*, vol. 8, pp. 87520–87534, 2020.
- [6] M. Khanafer and S. Shirmohammadi, "Survey of common methods to detect parking occupancy," *Sensors*, vol. 19, no. 16, p. 3568, 2019.
- [7] Republic Act No. 9292, *Electronics Engineering Law of the Philippines*, April 17, 2004.
- [8] Institute of Integrated Electrical Engineers of the Philippines, *Philippine Electrical Code (PEC)*, 2017 Edition.
- [9] S. Mobizt, "Firebase-ESP-Client library for Arduino," GitHub repository, accessed May 2026. [Online]. Available: <https://github.com/mobizt/Firebase-ESP-Client>
- [10] KiCad Development Team, *KiCad EDA Documentation v7.0*, 2024. [Online]. Available: <https://docs.kicad.org/>

Appendix A — ESP32 Firmware Source Code

The following is the complete source listing of `SmartParking.ino`, the firmware uploaded to the ESP32. The file is approximately 1,000 lines including comments. It is reproduced here verbatim.

```
[ PASTE FULL SmartParking.ino HERE ]
```

Appendix B — Web Application Source Code

The following is the complete source listing of `index.html`, the web application hosted on Firebase Hosting.

```
[ PASTE FULL index.html HERE ]
```

Appendix C — Bill of Materials with Costs

The complete bill of materials with unit costs and total project cost is reproduced from the project's BOM workbook.

[BOM table (or screenshot) — INSERT HERE — INSERT HERE]

Appendix D — High-Resolution Prototype Photographs

Photographs of the completed prototype, including detail views of the breadboard, the gate assembly, the slot daughterboards, and the LCD display.

[Photograph 1 — full prototype overview — INSERT HERE]

[Photograph 2 — controller breadboard close-up — INSERT HERE]

[Photograph 3 — gate assembly with servo and sensors — INSERT HERE]

[Photograph 4 — individual slot daughterboard — INSERT HERE]

[Photograph 5 — LCD display and web app side by side — INSERT HERE]